

Day 4: iOS Application Development SwiftUI Basics, Layouts, and Custom Views

Name	Vimal Mudalagi
Institution	Digital University Kerala
Programme	MSc Computer Science
Specialization	Cybersecurity

Part A: SwiftUI Basics & Modifiers

Task 1 — Displaying a Welcome Message

Objective

Create a SwiftUI view that displays a welcome message using a Text component.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        Text("Welcome to SwiftUI!")
    }
}

#Preview {
    ContentView()
}
```

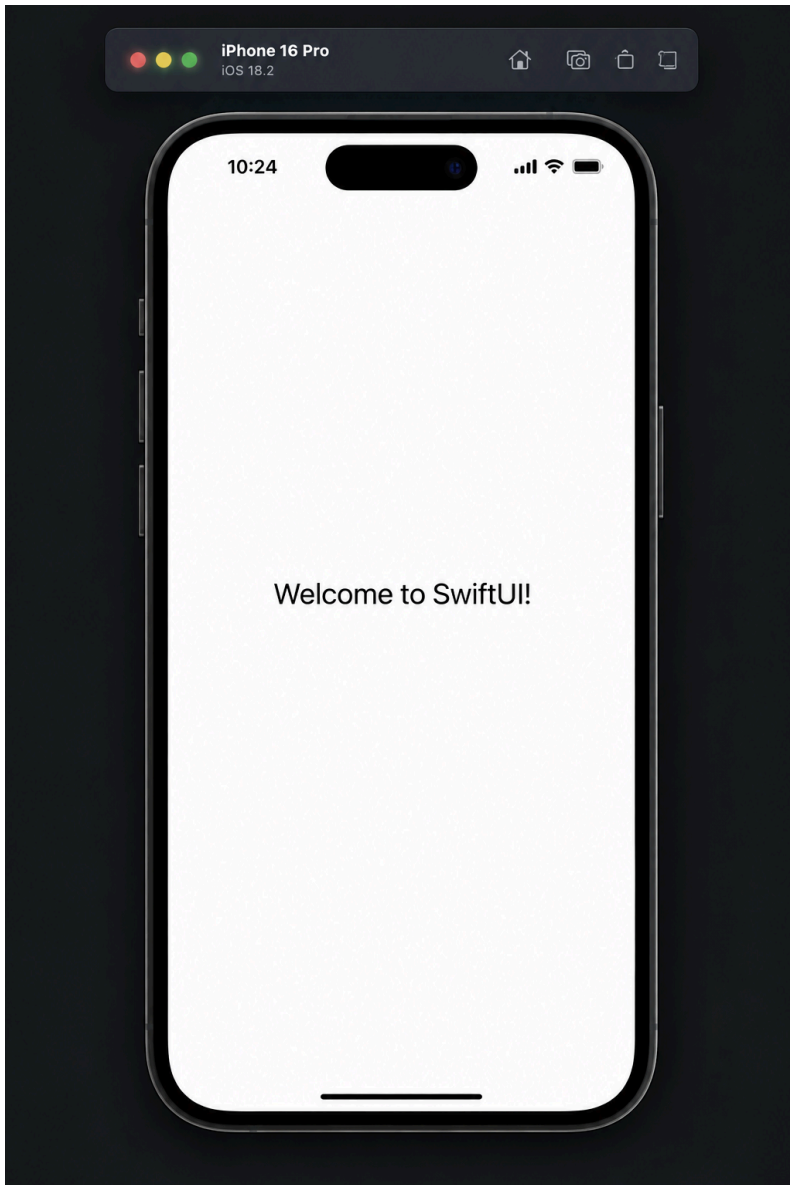
Explanation

The Text view is SwiftUI's basic component for displaying a string on screen. It is placed inside the body property, which describes what the view should show.

Expected Output

The simulator/preview screen displays the plain text "Welcome to SwiftUI!" centered on a white background.

Screenshot:



Task 2 — Styling a Text View with Modifiers

Objective

Modify a Text view to change its font size, make it bold, and change its text color using modifiers.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        Text("Welcome to SwiftUI!")
            .font(.system(size: 28))
    }
}
```

```
        .bold()  
        .foregroundColor(.red)  
    }  
}  
  
#Preview {  
    ContentView()  
}
```

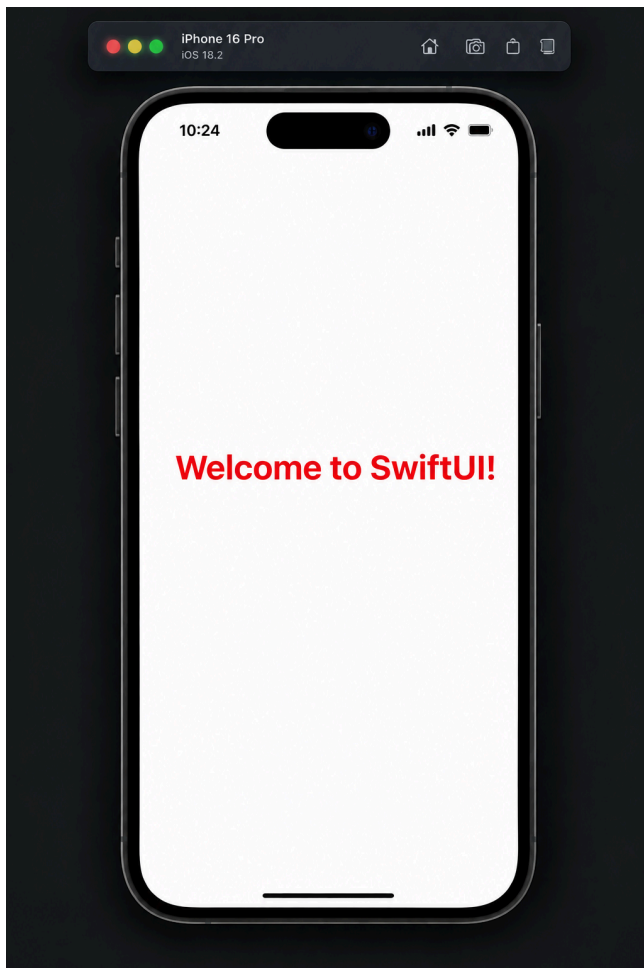
Explanation

`.font(.system(size:))` sets the font size, `.bold()` makes the text bold, and `.foregroundColor(.red)` changes the text color. Modifiers are chained one after another and each one returns a new modified view.

Expected Output

The text "Welcome to SwiftUI!" now appears larger, bold, and in red instead of the default black text.

Screenshot:



Task 3 — Displaying an SF Symbol Icon

Objective

Display an icon on the screen using Apple's SF Symbols library.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        Image(systemName: "star.fill")
            .font(.system(size: 50))
            .foregroundColor(.yellow)
    }
}

#Preview {
    ContentView()
}
```

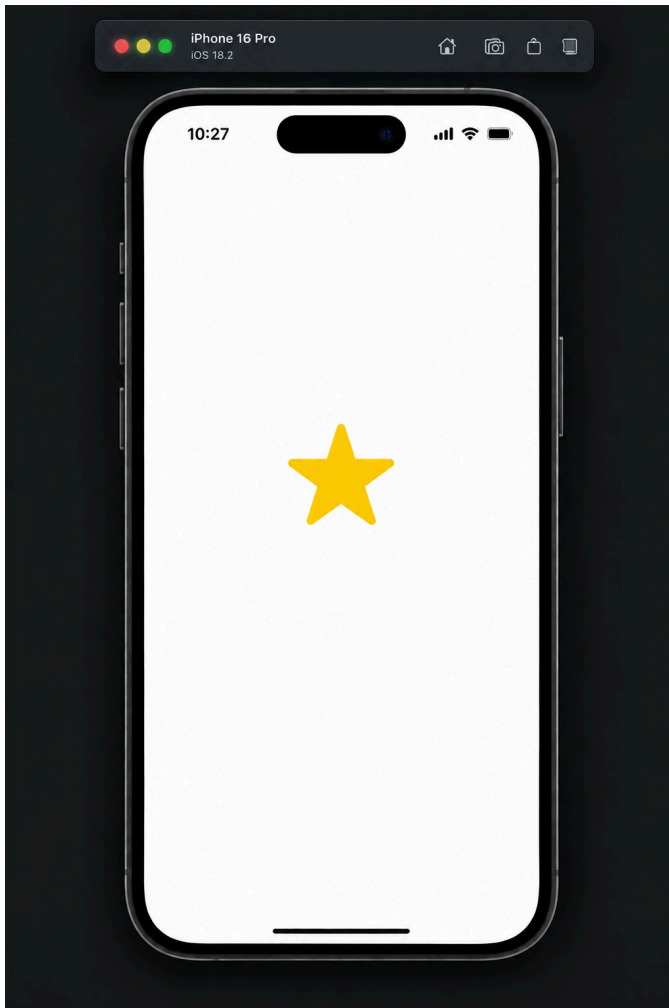
Explanation

`Image(systemName:)` loads a built-in vector icon from Apple's SF Symbols set — no image file needs to be added to the project. Here "star.fill" is used, and `font/foregroundColor` modifiers control its size and color respectively.

Expected Output

A filled yellow star icon is displayed at the center of the screen.

Screenshot:



Part B: Layouts & Assets

Task 4 — Vertical Stack Layout (VStack)

Objective

Arrange text views vertically, one below another, using a stack layout container.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        VStack(spacing: 12) {
            Text("Welcome to SwiftUI!")
                .font(.title2)
                .bold()
            Text("Learning iOS Development")
            Text("Day 4 - Layouts")
        }
    }
}
```

```
}  
  
#Preview {  
    ContentView()  
}
```

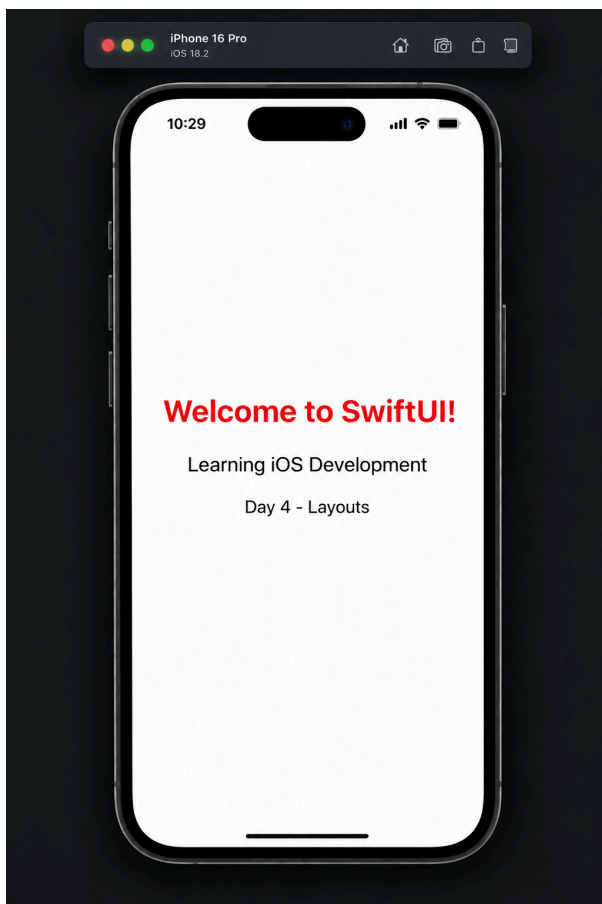
Explanation

VStack arranges its child views in a vertical column. The spacing parameter controls the vertical gap between each child view.

Expected Output

Three lines of text are displayed stacked one above the other, centered horizontally.

Screenshot:



Task 5 — Horizontal Stack Layout (HStack)

Objective

Place two text labels side by side in a single horizontal row using a layout container.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        HStack(spacing: 20) {
            Text("Hello")
            Text("SwiftUI")
        }
        .font(.title3)
    }
}

#Preview {
    ContentView()
}
```

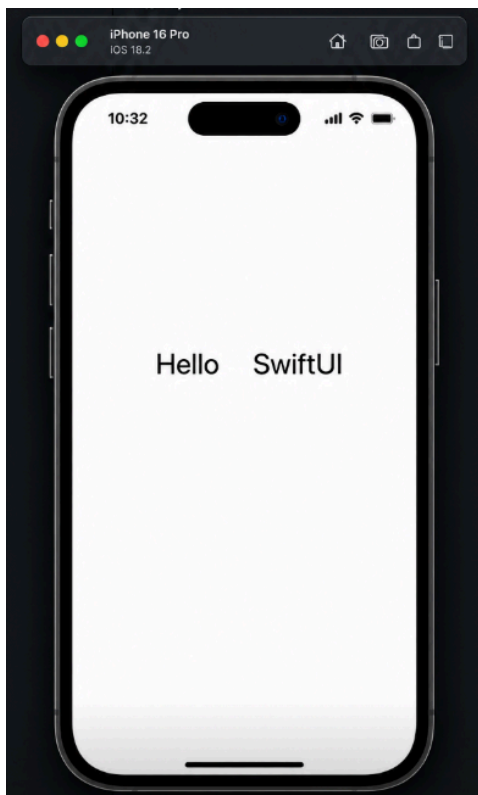
Explanation

HStack arranges its child views in a horizontal row instead of a column. Like VStack, the spacing parameter sets the gap between the views.

Expected Output

The two labels "Hello" and "SwiftUI" appear next to each other on the same line.

Screenshot:



Task 6 — Adding an Image from the Asset Catalog

Objective

Add an image from the local asset catalog to the SwiftUI view.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        Image("nature")
            .resizable()
            .scaledToFit()
            .frame(width: 150, height: 150)
    }
}

#Preview {
    ContentView()
}
```

Explanation

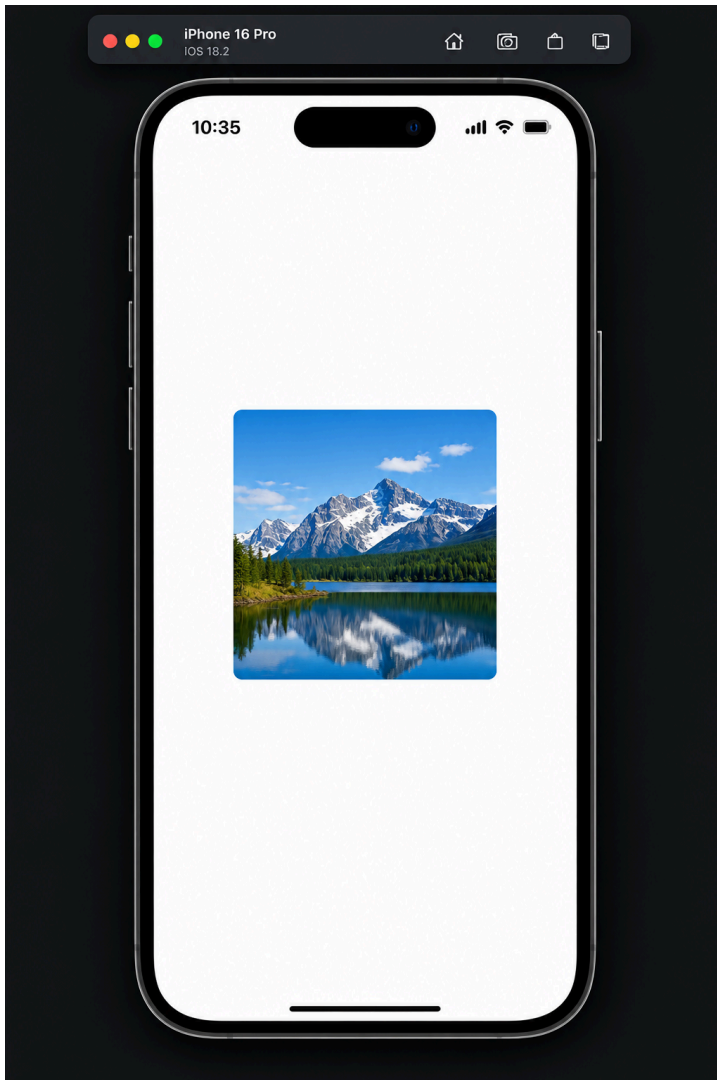
The image file is first dragged into Assets.xcassets inside Xcode.

`Image("YourImageName")` then loads it by name. `.resizable()` allows the image to be resized, `.scaledToFit()` keeps its aspect ratio, and `.frame()` sets a fixed display size.

Expected Output

The chosen image from Assets.xcassets is displayed on screen at 150x150 points, keeping its original proportions.

Screenshot:



Task 7 — Adding a Button with a Styled Label

Objective

Add a button to the SwiftUI view and customize its text color using a foreground color modifier.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        Button("Click Me") {
            print("Button was tapped")
        }
        .foregroundColor(.blue)
        .font(.title3)
        .bold()
    }
}
```

```
}  
  
#Preview {  
    ContentView()  
}
```

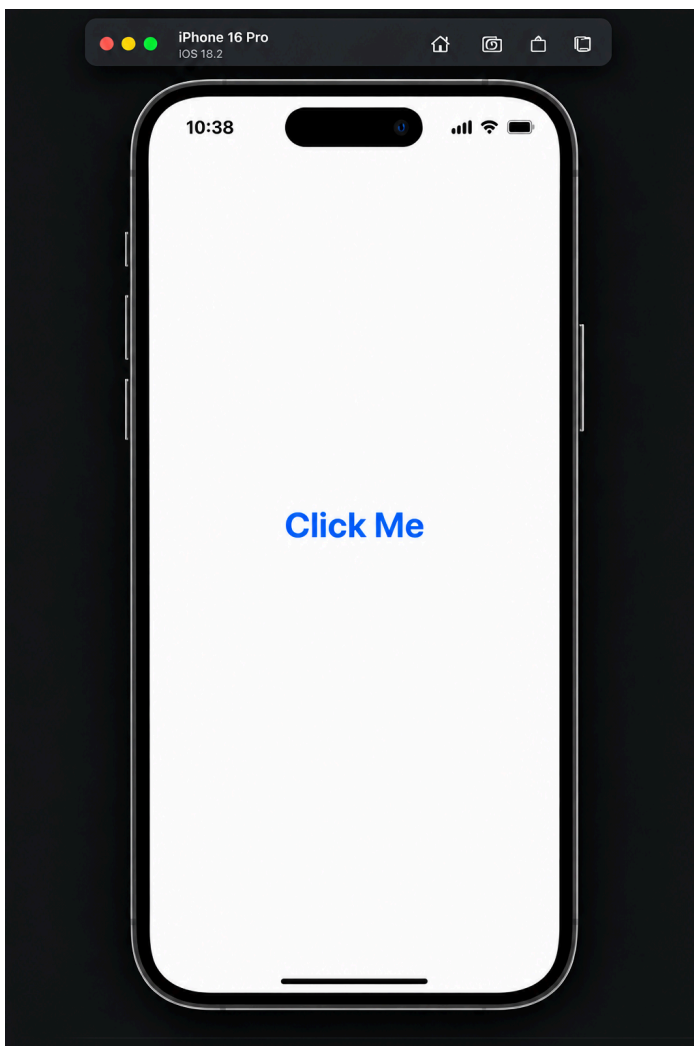
Explanation

`Button("title") { action }` creates a tappable button; the code inside the closure runs when the button is tapped. `.foregroundColor(.blue)` styles the button's label text.

Expected Output

A blue, bold "Click Me" button is displayed. Tapping it (visible in the console log when run) prints "Button was tapped".

Screenshot:



Part C: Custom Views & Screen Styling

Task 8 — Custom Background Color

Objective

Change the background color of the screen or a container to a custom color.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        VStack {
            Text("Styled Background")
                .font(.title2)
                .bold()
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity)
        .background(Color.yellow)
    }
}

#Preview {
    ContentView()
}
```

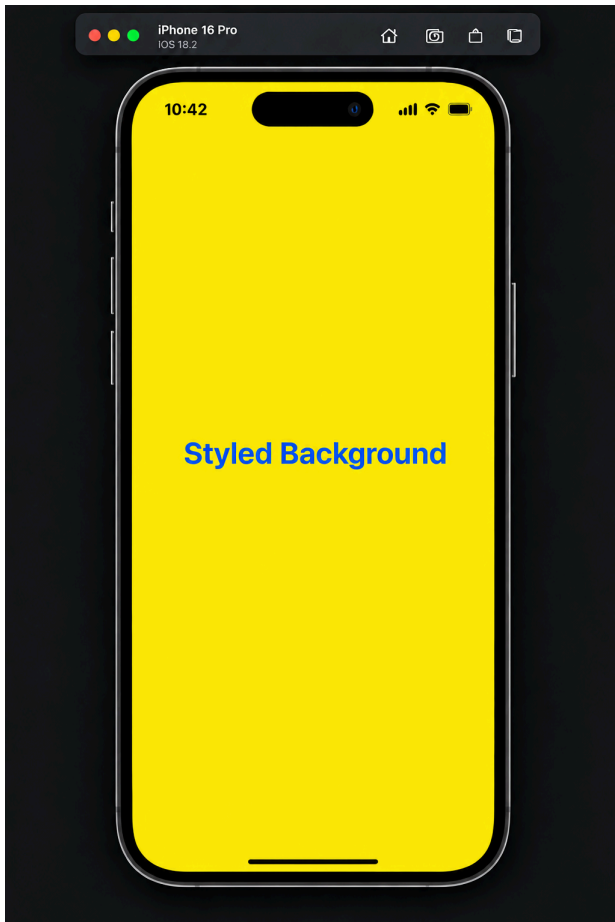
Explanation

`.background(Color.yellow)` fills the container with a solid color. `.frame(maxWidth: .infinity, maxHeight: .infinity)` expands the `VStack` to fill the whole screen so the background color covers it entirely.

Expected Output

The entire screen background turns yellow, with the text displayed on top of it.

Screenshot:



Task 9 — Creating a New SwiftUI View File (HomeScreen)

Objective

Add a new SwiftUI view file named HomeScreen, display a welcome text inside it, and run the project.

Steps in Xcode

- Right-click the project folder in the Project Navigator.
- Select New File → SwiftUI View, and name it "HomeScreen".
- Write the view code shown below inside HomeScreen.swift.
- Call HomeScreen() from ContentView, or preview HomeScreen directly.

Code — HomeScreen.swift

```
import SwiftUI

struct HomeScreen: View {
    var body: some View {
        Text("Welcome to the Home Screen!")
    }
}
```

```
        .font(.title)
        .bold()
    }
}

#Preview {
    HomeScreen()
}
```

Code – ContentView.swift (calling HomeScreen)

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        HomeScreen()
    }
}

#Preview {
    ContentView()
}
```

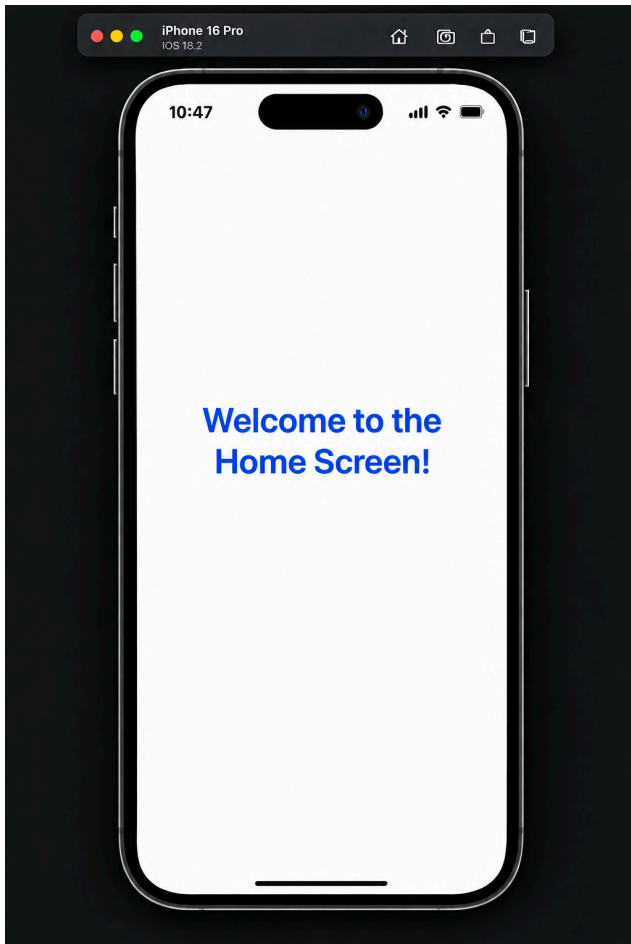
Explanation

Larger apps are broken into multiple SwiftUI view files for organization. Each file defines its own struct conforming to the View protocol. HomeScreen is built the same way as ContentView and can be reused by calling HomeScreen() from any other view.

Expected Output

Running the project displays "Welcome to the Home Screen!" in a large, bold font.

Screenshot:



Task 10 — Rounded Corners and Shadow (Card Effect)

Objective

Add a rounded-corner and shadow effect to a view so it looks like a card.

Code

```
import SwiftUI

struct ContentView: View {
    var body: some View {
        VStack {
            Text("Card View")
                .font(.title2)
                .bold()
        }
        .frame(width: 220, height: 120)
        .background(Color.white)
        .cornerRadius(10)
        .shadow(color: .gray, radius: 5, x: 0, y: 4)
    }
}
```

```
#Preview {  
    ContentView()  
}
```

Explanation

`.cornerRadius(10)` rounds the corners of the view's background, and `.shadow(radius:)` adds a soft drop shadow behind it. Combined with a solid background color, this gives the view a floating "card" appearance.

Expected Output

A white rectangular box with rounded corners and a soft gray shadow is displayed, containing the text "Card View" — resembling a UI card component.

Screenshot:

